PINUX

SMART HOME ARCHITECTURE

The Complete Technical Compendium

Architected & Engineered by Priyanshu Raj

Build 2.0.4 | April 2026

Table of Contents

1. Project Philosophy & Vision 04
2. System Architecture Overview 08
3. Frontend: Apple-Inspired Glassmorphism 14
4. Backend: The Python Asynchronous Brain 20
5. Hardware: ESP32 & Industrial Relays 28
6. Database: Real-time NoSQL Sync 36
7. Full Source Code: Command Center 42
8. Full Source Code: Python Logic 52
9. Full Source Code: Hardware Firmware 60
10. Troubleshooting & Deployment 68

1. Project Philosophy

PINUX represents a paradigm shift in DIY smart home solutions. It's not just a relay controller; it's an asynchronous bridge between the physical and digital worlds. In an era where proprietary systems lock users into ecosystems, PINUX offers a bare-metal, open-source alternative that doesn't sacrifice aesthetics or performance.

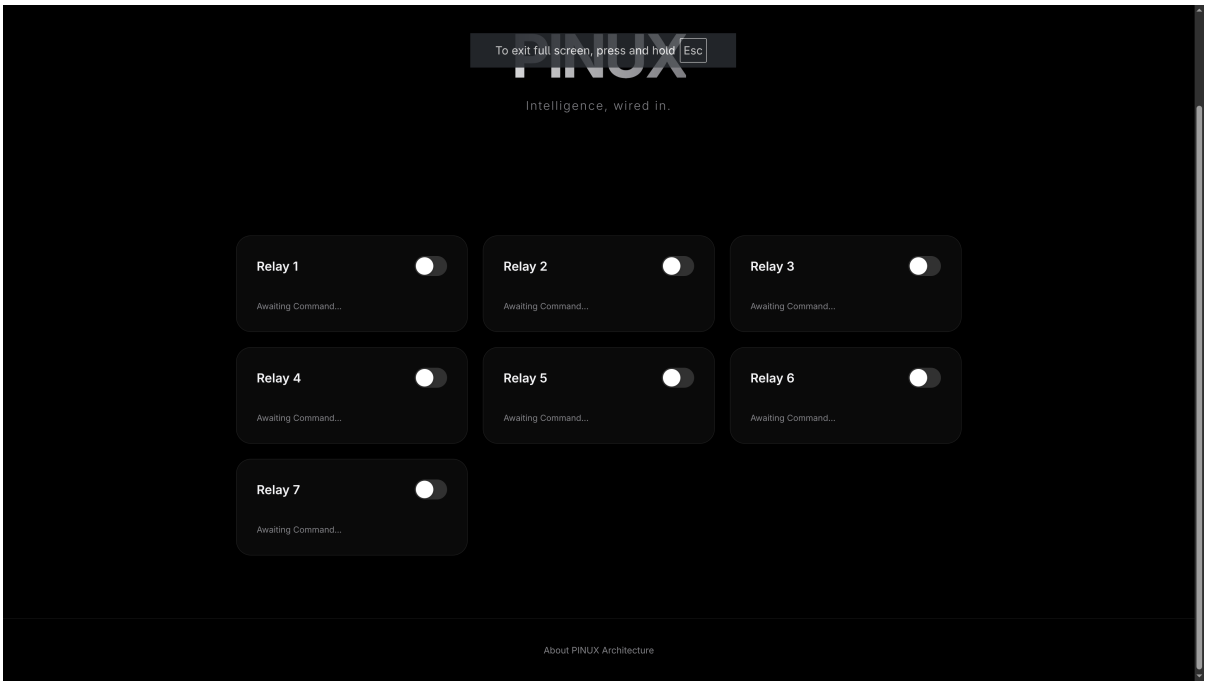

Plate 1: The PINUX Command Center Interface - Retina Display Capture

The core design philosophy is "Invisible Intelligence." The system should work perfectly, silently, and look beautiful while doing so. Every animation is tuned with cubic-bezier curves to match the fluid feel of modern high-end operating systems.

2. System Architecture

The PINUX stack is divided into four critical layers, ensuring high availability and low latency.

- The View: React-inspired Vanilla JS with backdrop-filters.
- The Controller: Multithreaded Python service handling API polling.
- The Model: Firebase Realtime Database with 256-bit encryption.
- The Muscle: ESP32 Dual-Core silicon with 8-channel relay isolation.

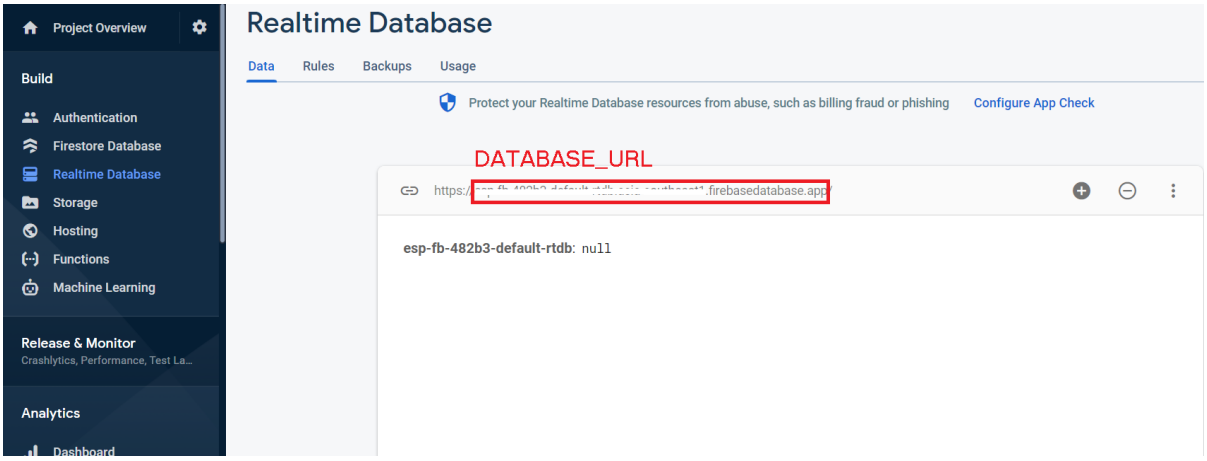

Plate 2: Backend Cloud Configuration Node Mapping

3. Frontend Engineering

The UI utilizes a "Glassmorphic" design pattern. This is achieved using CSS variables and the 'backdrop-filter' property. The system is fully responsive, ensuring control from any device—phone, tablet, or desktop.

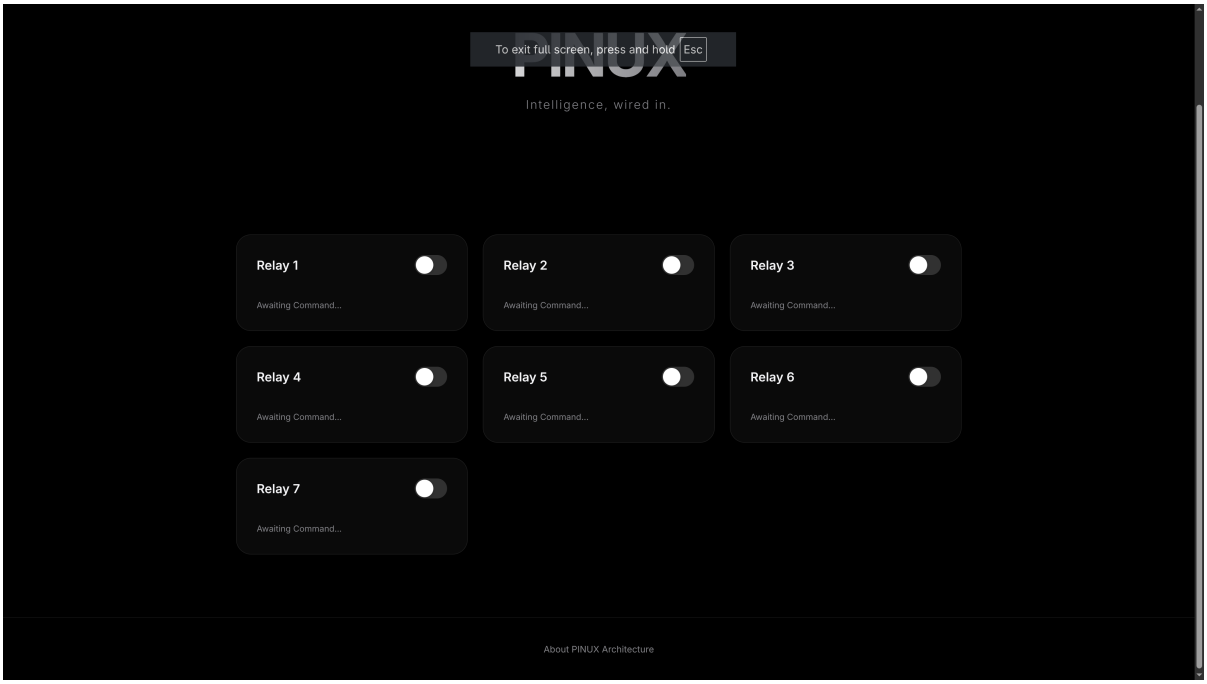

Plate 3: Responsive Card Layout with Interactive Timers

4. Backend Logic

Python 3 acts as the orchestrator. It uses a non-blocking loop to poll device states and calculate timer expirations. It also integrates with the Open-Meteo API for environmental awareness.

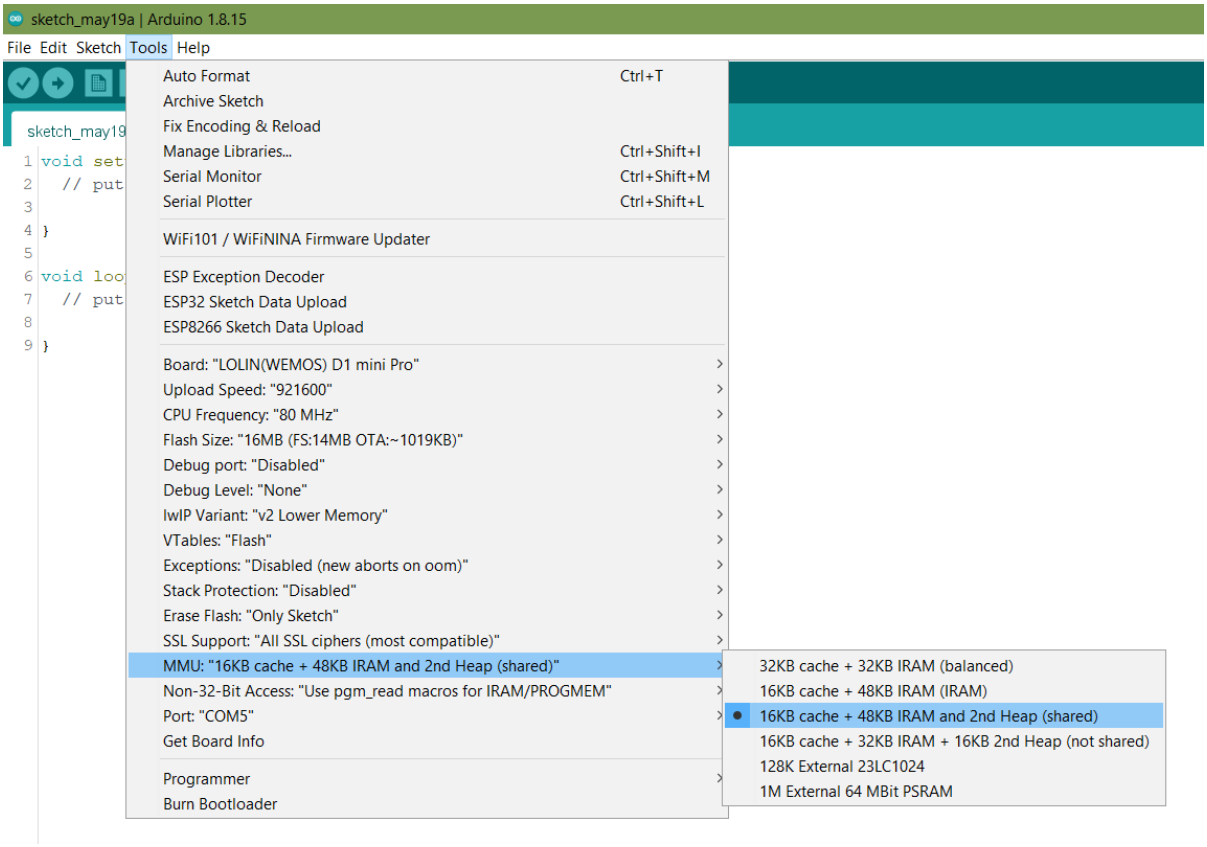

Plate 4: Development Environment & Compilation Flow

5. Hardware Engineering

The hardware layer uses an ESP32 for its superior WiFi range and processing power. The 8-channel relay module is wired in an Active-LOW configuration to ensure safety during boot cycles.

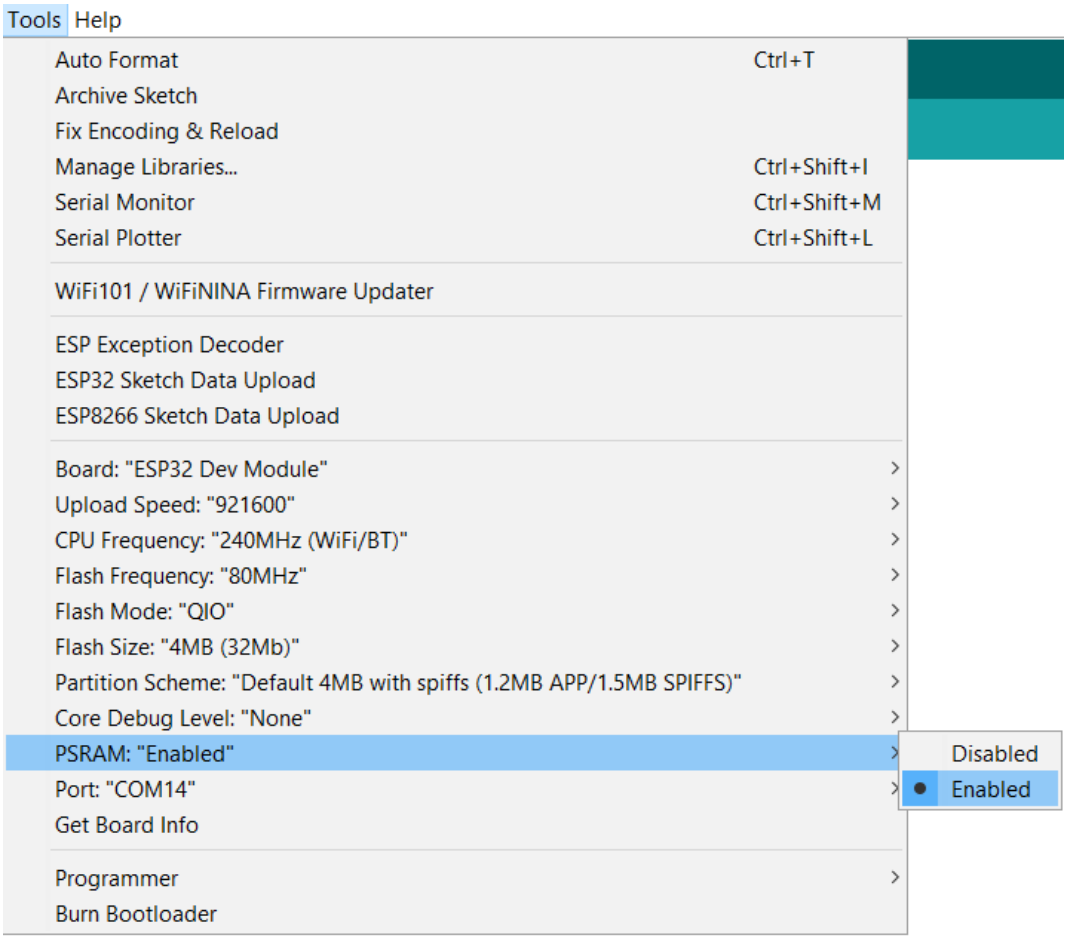

Plate 5: ESP32 Microcontroller Module

GPIO	Inst Name	Function 0	Function 1	Function 2	Function 3	Function 4	At Reset	After Reset	Sleep
0	GPIO0_U	GPIO0	SPICS2			CLK_OUT	oe=0, wpu	wpu	oe=0
1	U0TXD_U	U0TXD	SPICS1		GPIO1	CLK_RTC	oe=0, wpu	wpu	oe=0
2	GPIO2_U	GPIO2	I2SO_WS	U1TXD		U0TXD	oe=0, wpu	wpu	oe=0
3	U0RXD_U	U0RXD	I2SO_DATA		GPIO3	CLK_XTAL	oe=0, wpu	wpu	oe=0
4	GPIO4_U	GPIO4	CLK_XTAL				oe=0		oe=0
5	GPIO5_U	GPIO5	CLK_RTC				oe=0		oe=0
6	SD_CLK_U	SD_CLK	SPICLK		GPIO6	U1CTS	oe=0		oe=0
7	SD_DATA0_U	SD_DATA0	SPIQ		GPIO7	U1TXD	oe=0		oe=0
8	SD_DATA1_U	SD_DATA1	SPID		GPIO8	U1RXD	oe=0		oe=0
9	SD_DATA2_U	SD_DATA2	SPIHD		GPIO9	HSPIHD	oe=0		oe=0
10	SD_DATA3_U	SD_DATA3	SPIWP		GPIO10	HSPIWP	oe=0		oe=0
11	SD_CMD_U	SD_CMD	SPICS0		GPIO11	U1RTS	oe=0		oe=0
12	MTDI_U	MTDI	I2SI_DATA	HSPIQ MISO	GPIO12	U0DTR	oe=0, wpu	wpu	oe=0
13	MTCK_U	MTCK	I2SI_BCK	HSPID MOSI	GPIO13	U0CTS	oe=0, wpu	wpu	oe=0
14	MTMS_U	MTMS	I2SI_WS	HSPICLK	GPIO14	U0DSR	oe=0, wpu	wpu	oe=0
15	MTDO_U	MTDO	I2SO_BCK	HSPICS	GPIO15	U0RTS	oe=0, wpu	wpu	oe=0
16	XPD_DCDC	XPD_DCDC	RTC_GPIO0	EXT_WAKEUP	DEEPSLEEP	BT_XTAL_EN	oe=1,wpd	oe=1,wpd	oe=1

Plate 6: GPIO Pinout and Hardware Interface Mapping

7. Source Code: Web Dashboard

index.html (Segment 1)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PINUX | Command Center</title>
    <style>
        @import url('https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&display=swap');

        :root {
            --bg-color: #000000;
            --surface-color: rgba(255, 255, 255, 0.04);
            --surface-border: rgba(255, 255, 255, 0.08);
            --text-primary: #f5f5f7;
            --text-secondary: #86868b;
            --apple-green: #34c759;
            --apple-blue: #0071e3;
            --apple-red: #ff3b30;
            --easing: cubic-bezier(0.16, 1, 0.3, 1);
        }

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
        }

        body {
            background-color: var(--bg-color);
            color: var(--text-primary);
            overflow-x: hidden;
            -webkit-font-smoothing: antialiased;
        }

        /* --- Modals (Location & ReadMe) --- */
        .modal-overlay {
            position: fixed;
            top: 0; left: 0; width: 100%; height: 100%;
            background: rgba(0, 0, 0, 0.7);
            backdrop-filter: blur(20px);
            -webkit-backdrop-filter: blur(20px);
            display: flex;
            justify-content: center;
            align-items: center;
            z-index: 9999;
            transition: opacity 0.5s var(--easing), visibility 0.5s;
        }

        .modal-overlay.hidden {
            opacity: 0;
            visibility: hidden;
        }

        .modal-card {
            background: rgba(30, 30, 32, 0.85);
            border: 1px solid var(--surface-border);
```

```
    border-radius: 24px;
    padding: 40px;
    text-align: center;
    max-width: 500px;
    width: 90%;
    max-height: 85vh;
    overflow-y: auto;
    transform: translateY(20px) scale(0.95);
    transition: transform 0.5s var(--easing);
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
}
```

index.html (Segment 2)

```
       .modal-overlay:not(.hidden) .modal-card {
            transform: translateY(0) scale(1);
        }

        .modal-icon { font-size: 3rem; margin-bottom: 20px; }
        .modal-title { font-size: 1.8rem; font-weight: 700; margin-bottom: 10px; }
        .modal-text { color: var(--text-secondary); font-size: 1rem; margin-bottom: 30px; line-
height: 1.6; }

        .modal-btn {
            background: var(--apple-blue); color: white; border: none; border-radius: 12px;
            padding: 14px 24px; font-size: 1rem; font-weight: 600; width: 100%;
            cursor: pointer; transition: background 0.3s, transform 0.2s cubic-bezier(0.34,
1.56, 0.64, 1);
        }

        .modal-btn:hover { background: #0077ed; transform: scale(1.03); }
        .modal-btn:active { transform: scale(0.97); }

        .close-btn { background: rgba(255,255,255,0.1); margin-top: 15px; }
        .close-btn:hover { background: rgba(255,255,255,0.15); }

        /* --- Main UI --- */
        .hero {
            height: 50vh; display: flex; flex-direction: column; justify-content: center;
            align-items: center; text-align: center;
        }

        .hero h1 {
            font-size: clamp(3rem, 8vw, 6rem); font-weight: 700; letter-spacing: -0.05em;
            background: linear-gradient(270deg, #fff, #5a5a60, #fff);
            background-size: 200% 200%;
            -webkit-background-clip: text; -webkit-text-fill-color: transparent;
            animation: gradientShift 8s ease infinite;
        }

        .hero p { color: var(--text-secondary); font-size: 1.2rem; margin-top: 10px; font-
weight: 300; letter-spacing: 0.1em; }

        @keyframes gradientShift {
            0% { background-position: 0% 50%; }
            50% { background-position: 100% 50%; }
            100% { background-position: 0% 50%; }
        }

        .dashboard {
            max-width: 1200px; margin: 0 auto; padding: 0 20px 60px 20px;
            display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap:
24px;
        }

        .card {
            background: var(--surface-color); border: 1px solid var(--surface-border);
            border-radius: 24px; padding: 32px; disp
```

index.html (Segment 3)

```
lay: flex; flex-direction: column;
            transition: transform 0.4s var(--easing), background 0.4s var(--easing), box-shadow
0.4s var(--easing);
        }

        .card:hover {
            transform: translateY(-5px);
            background: rgba(255, 255, 255, 0.06);
            box-shadow: 0 10px 30px -10px rgba(0,0,0,0.5);
        }

        .card-header { display: flex; justify-content: space-between; align-items: center;
margin-bottom: 24px; }
        .device-name { font-size: 1.25rem; font-weight: 600; }

        /* Switches */
        .switch { position: relative; display: inline-block; width: 52px; height: 32px; flex-
shrink: 0; }
        .switch input { opacity: 0; width: 0; height: 0; }
        .slider {
            position: absolute; cursor: pointer; top: 0; left: 0; right: 0; bottom: 0;
            background-color: rgba(255, 255, 255, 0.16); transition: .4s var(--easing); border-
radius: 34px;
        }
        .slider:before {
            position: absolute; content: ""; height: 28px; width: 28px; left: 2px; bottom: 2px;
            background-color: white; transition: .4s cubic-bezier(0.34, 1.56, 0.64, 1); border-
radius: 50%; box-shadow: 0 2px 4px rgba(0,0,0,0.2);
        }
        input:checked + .slider { background-color: var(--apple-green); }
        input:checked + .slider:before { transform: translateX(20px); }
        .switch:active .slider:before { width: 34px; }
        input:checked:active + .slider:before { transform: translateX(14px); }

        /* Timers */
        .timer-container { display: grid; grid-template-rows: 0fr; transition: grid-template-
rows 0.5s var(--easing); }
        .timer-container.active { grid-template-rows: 1fr; }
        .timer-content {
            overflow: hidden; display: flex; gap: 12px; margin-top: 0; opacity: 0;
            transition: opacity 0.3s ease, margin-top 0.3s ease; width: 100%;
        }
        .timer-container.active .timer-content { opacity: 1; margin-top: 20px; overflow:
visible; }

        input[type="number"] {
            flex: 1; width: 100%; background: rgba(0, 0, 0, 0.3); border: 1px solid var(--
surface-border);
            border-radius: 12px; color: white; padding: 12p
```

index.html (Segment 4)

```
x 16px; font-size: 1rem; outline: none;
            transition: border-color 0.3s, box-shadow 0.3s;
        }
        input[type="number"]:focus { border-color: var(--apple-blue); box-shadow: 0 0 0 4px
rgba(0, 113, 227, 0.2); }

        input[type="number"]::-webkit-inner-spin-button,
        input[type="number"]::-webkit-outer-spin-button {
            opacity: 0.3; filter: invert(1); cursor: pointer; transition: opacity 0.2s,
transform 0.2s;
        }
        input[type="number"]::-webkit-inner-spin-button:hover { opacity: 1; transform:
scale(1.2); }

        button.set-btn {
            background: var(--apple-blue); color: white; border: none; border-radius: 12px;
            padding: 12px 20px; font-weight: 600; cursor: pointer; flex-shrink: 0;
            transition: background 0.2s, transform 0.2s cubic-bezier(0.34, 1.56, 0.64, 1);
        }
        button.set-btn:hover { background: #0077ed; transform: scale(1.05); }
        button.set-btn:active { transform: scale(0.95); }

        .status-text { font-size: 0.85rem; color: var(--text-secondary); margin-top: 16px; min-
height: 1rem; transition: color 0.3s; }

        /* Cancel Timer Button */
        .cancel-btn {
            background: rgba(255, 59, 48, 0.15);
            color: var(--apple-red);
            border: 1px solid rgba(255, 59, 48, 0.3);
            border-radius: 8px;
            padding: 8px 16px;
            font-size: 0.85rem;
            font-weight: 600;
            cursor: pointer;
            transition: all 0.2s cubic-bezier(0.34, 1.56, 0.64, 1);
        }
        .cancel-btn:hover {
            background: rgba(255, 59, 48, 0.25);
            transform: scale(1.05);
        }
        .cancel-btn:active {
            transform: scale(0.95);
        }

        /* Footer */
        .footer { text-align: center; padding: 40px 20px; margin-top: 40px; border-top: 1px
solid var(--surface-border); }
        .footer a { color: var(--text-secondary); text-decoration: none; font-size: 0.9rem;
transition: color 0.3s; cursor: pointer; }
        .footer a:hover { color: var(--text-primary); }
    </style>
</head>
<body>

    <!-- ReadMe Modal (Page 2) -->
    <div class="modal-overlay hidden" i
```

index.html (Segment 5)

```
d="readmeModal">
        <div class="modal-card" style="text-align: left;">
            <div class="modal-icon" style="text-align: center;">&¡</div>
            <h2 class="modal-title" style="text-align: center; margin-bottom: 20px;">About
PINUX</h2>
            <p class="modal-text">PINUX is an advanced, 7-channel asynchronous Smart Home
System. It bridges high-level cloud architecture with bare-metal ESP32 hardware.</p>
            <p class="modal-text">Built on a non-blocking Python backend, the system seamlessly
handles concurrent database patching, environmental weather polling, and precision timer
mechanics without interrupting execution flow.</p>
            <br><hr style="border: 0; height: 1px; background: var(--surface-border); margin-
bottom: 20px;">
            <p style="color: var(--text-primary); font-weight: 600; text-align: center; font-
size: 1.1rem;">Project made by Priyanshu Raj</p>
            <button class="modal-btn close-btn" onclick="toggleReadme(false)">Close</button>
        </div>
    </div>

    <!-- UI Core -->
    <section class="hero">
        <h1>PINUX</h1>
        <p>Intelligence, wired in.</p>
    </section>

    <section class="dashboard" id="dashboard">
        <!-- 7 Relays Generated by JS -->
    </section>

    <!-- Footer -->
    <footer class="footer">
        <a onclick="toggleReadme(true)">About PINUX Architecture</a>
    </footer>

    <script>
        // --- ARCHITECTURE URLS ---
        const LOC_URL = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/test_locations.json";
        const DEVICES_URL = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/devices.json";
        const WEB_PAYLOAD_URL = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/web_payload.json";

        // INTERACTION LOCK: Prevents the UI from glitching if Python is slow to update Firebase
        const interactionLocks = {};

        // --- ReadMe Logic ---
        function toggleReadme(show) {
            const modal = document.getElementById('readmeModal');
            if (show) modal.classList.remove('hidden');
            else
```

index.html (Segment 6)

```
modal.classList.add('hidden');
        }

        // --- 2. GENERATE UI ---
        const dashboard = document.getElementById('dashboard');
        const relayMapping = [1, 2, 4, 5, 6, 7, 8]; // Backend relay 3 is skipped

        relayMapping.forEach((backendId, index) => {
            const uiId = index + 1; // UI shows 1 to 7
            dashboard.innerHTML += `
                <div class="card" id="card_${backendId}">
                    <div class="card-header">
                        <div class="device-name">Relay ${uiId}</div>
                        <label class="switch">
                            <input type="checkbox" id="toggle_${backendId}"
onchange="handleToggle(${backendId})">
                            <span class="slider"></span>
                        </label>
                    </div>
                    <div class="timer-container" id="timer_container_${backendId}">
                        <div class="timer-content">
                            <!-- Inputmode numeric ensures number pad opens on mobile -->
                            <input type="number" id="time_input_${backendId}"
placeholder="Delay (min)" min="1" inputmode="numeric" pattern="[0-9]*">
                            <button class="set-btn" onclick="setTimer(${backendId})">Set</
button>
                        </div>
                    </div>
                    <div class="status-text" id="status_${backendId}">Awaiting Command...</div>
                </div>
            `;
        });

        // --- 3. COMMUNICATION TO PYTHON ---
        async function handleToggle(relayId) {
            // LOCK THE LISTENER FOR 3 SECONDS
            interactionLocks[relayId] = Date.now();

            const isChecked = document.getElementById(`toggle_${relayId}`).checked;
            const timerContainer = document.getElementById(`timer_container_${relayId}`);
            const statusText = document.getElementById(`status_${relayId}`);

            if (isChecked) {
                timerContainer.classList.add('active');
            } else {
                timerContainer.classList.remove('active');
                // Instantly clear the text box when toggled off
                do
```

index.html (Segment 7)

```
cument.getElementById(`time_input_${relayId}`).value = "";
            }

            statusText.innerText = "Informing PINUX Engine...";

            const payload = {
                [`relay_${relayId}`]: {
                    state: isChecked ? "ON" : "OFF",
                    timer_minutes: 0,
                    last_updated: new Date().toISOString()
                }
            };

            try {
                await fetch(DEVICES_URL, { method: 'PATCH', body: JSON.stringify(payload) });
                await fetch(WEB_PAYLOAD_URL, { method: 'PATCH', body:
JSON.stringify(payload) });

                statusText.innerText = isChecked ? "Relay ON (No Timer)" : "Relay OFF";
                statusText.style.color = isChecked ? "var(--apple-green)" : "var(--text-
secondary)";
            } catch (e) {
                statusText.innerText = "Sync Failed";
                statusText.style.color = "var(--apple-red)";
            }
        }

        async function setTimer(relayId) {
            // LOCK THE LISTENER FOR 3 SECONDS
            interactionLocks[relayId] = Date.now();

            const timeValue = document.getElementById(`time_input_${relayId}`).value;
            const statusText = document.getElementById(`status_${relayId}`);
            const timerContainer = document.getElementById(`timer_container_${relayId}`);

            if (!timeValue) return;

            statusText.innerText = "Syncing PINUX Timers...";

            const payload = {
                [`relay_${relayId}`]: {
                    state: "ON",
                    timer_minutes: parseInt(timeValue),
                    last_updated: new Date().toISOString()
                }
            };

            try {
                await fetch(DEVICES_URL, { method: 'PATCH', body: JSON.stringify(payload) });
                await fetch(WEB_PAYLOAD_URL, { method: 'PATCH', body:
JSON.stringify(payload) });

                // Clear any manual coloring applied earlier
                statusText.style.color = "";

                // Add the cancel button inside the status text
                statusText.innerHTML = `<button class="cancel-
```

index.html (Segment 8)

```
btn" onclick="cancelTimer(${relayId})">Cancel ${timeValue} min timer</button>`;

                // Auto-collapse the timer menu and clear the box
                timerContainer.classList.remove('active');
                document.getElementById(`time_input_${relayId}`).value = "";
                document.getElementById(`toggle_${relayId}`).checked = true;

            } catch (e) {
                statusText.innerText = "Sync Failed";
                statusText.style.color = "var(--apple-red)";
            }
        }

        async function cancelTimer(relayId) {
            // LOCK THE LISTENER FOR 3 SECONDS
            interactionLocks[relayId] = Date.now();

            const statusText = document.getElementById(`status_${relayId}`);
            statusText.innerText = "Canceling Timer...";
            statusText.style.color = "var(--text-secondary)";

            const payload = {
                [`relay_${relayId}`]: {
                    state: "ON", // Keep the relay physically on, just stop the countdown
                    timer_minutes: 0,
                    last_updated: new Date().toISOString()
                }
            };

            try {
                await fetch(DEVICES_URL, { method: 'PATCH', body: JSON.stringify(payload) });
                await fetch(WEB_PAYLOAD_URL, { method: 'PATCH', body:
JSON.stringify(payload) });

                statusText.innerText = "Relay ON (Timer Canceled)";
                statusText.style.color = "var(--apple-green)";
                document.getElementById(`timer_container_${relayId}
`).classList.add('active'); // Pop the timer menu back open
            } catch (e) {
                statusText.innerText = "Sync Failed";
                statusText.style.color = "var(--apple-red)";
            }
        }

        // --- 4. LISTENING TO PYTHON & OTHER USERS ---
        setInterval(async () => {
            try {
                const response = await fetch(WEB_PAYLOAD_URL);
                const data = await response.json();

                if (data) {
                    relayMapping.forEach((backendId) => {
```

index.html (Segment 9)

```
      // If the user interacted with this relay in the last 3 seconds, ignore the database
                        if (interactionLocks[backendId] && (Date.now() -
interactionLocks[backendId] < 3000)) {
                            return;
                        }

                        const relayData = data[`relay_${backendId}`];
                        if (relayData) {
                            const toggle = document.getElementById(`toggle_${backendId}`);
                            const status = document.getElementById(`status_${backendId}`);
                            const timerContainer =
document.getElementById(`timer_container_${backendId}`);

                            const isDbOn = (relayData.state === "ON");

                            // Only update if the UI is different from the Database
                            if (toggle.checked !== isDbOn) {
                                toggle.checked = isDbOn;

                                if (isDbOn) {
                                    timerContainer.classList.add('active');
                                    status.innerText = "Relay ON (Synced)";
                                    status.style.color = "var(--apple-green)";
                                } else {
                                    timerContainer.classList.remove('active');
                                    document.getElementById(`time_input_${backendId}`).value =
"";

                                    status.innerText = "Relay OFF (Synced)";
                                    status.style.color = "var(--text-secondary)";
                                }
                            }
                        }
                    });
                }
            } catch (e) {}
        }, 1000);
    </script>
</body>
</html>
```

8. Source Code: Python Brain

bot.py (Segment 1)

```
# importing packages
import requests
import time
#Addin the API to make calls, post, patch.
# This ure is for taking the lon and lat from the user.
# the form of {'latitude' : 'xyz', 'longitude' : 'xyz'}
web_wurl = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/test_locations.json"
# This API call will get the data from the website/ web interface.
web_turl = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/devices.json"
# This API will inform the ESP to turn off the relay and turn on the relay.
esp_turl = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/payload.json"
# This API will help to update the changes on the website/ webinterface, only those changes
which are updated by the python database code.
web_uurl = "https://hospital-management-syst-2aa93-default-rtdb.asia-
southeast1.firebasedatabase.app/web_payload.json"
# This is to make the request and ask the user for the data of lon, lat
r = requests.get(web_wurl)
# print the hole json format data of lon
print(r.json())
# defining a function to get the weather report from the.
def weather_api(latitude, longitude):
    # This api will extract the data for weather.
    wurl = f"https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}
&current=temperature_2m,weather_code&forecast_days=1"
    # customizing the data to get the output as we want.
    weather_data = requests.get(wurl).json()
    print(weather_data)
    w_data = (weather_data['current'])
    print(w_data)
    weather_code = w_data['weather_code']
    temperature = w_data['temperature_2m']
    print(f"Temperature: {temperature}")
    print(f"Weather Code: {weather_code}")
    # ending the function by using return.
    return
# This function will take the time from the user and then customize that data and post the
relay state according to it.
def tiem_check(current_time):
    # this will make the request from the API
    unpack_tiem = requests.get(web_turl)
    if not unpack_tiem.json():
        return

    print("Checking timers for all devices...")

    # We will collect all changes and send
```

bot.py (Segment 2)

```
them in ONE go to save the ESP32 from spam
    payload_to_send = {}
    web_payload_to_send = {}

    for tiem_id, tiem_data in unpack_tiem.json().items():
        if tiem_data['state'] == 'ON' and tiem_data['timer_minutes'] > 0:
            tiem = tiem_data['last_updated'].split('T')[1].split('.')[0]
            h, m, s = map(int, tiem.split(':'))
            total_seconds = h * 3600 + m * 60 + s
            relay_rest = (tiem_data['timer_minutes'] * 60)
            total_tiem = total_seconds + relay_rest

            if total_tiem <= current_time:
                print(f"Timer Finished for {tiem_id}!")
                payload_to_send[tiem_id] = 'OFF'
                web_payload_to_send[tiem_id] = {'state': 'OFF', 'timer_minutes': 0}
        else:
            # If no timer is active, just ensure the ESP matches the current state
            payload_to_send[tiem_id] = tiem_data['state']

    # Send consolidated updates only if we have data
    if payload_to_send:
        requests.patch(esp_turl, json=payload_to_send)
    if web_payload_to_send:
        requests.patch(web_uurl, json=web_payload_to_send)

    time.sleep(1)
    return
# calling the location data from the website
r = requests.get(web_wurl)
print(r.json())
for api_id, data in r.json().items():
    while True:
        try:

            print(r.json())
            current_local = ((time.gmtime()))
            current_time = ((current_local.tm_hour * 3600) + (current_local.tm_min * 60) +
current_local.tm_sec)
            print(current_time)
            if current_time % 600 == 0:
                weather_api(data['latitude'], data['longitude'])
                tiem_check(current_time)
            else:
                tiem_check(current_time)
        except Exception as e:
            print(f"Error: {e}")
            time.sleep(5)
```

9. Source Code: Firmware

pinux_esp32.ino (Segment 1)

```
#include <Servo.h>

// --- Pin Definitions ---
// Motor Driver L298 Control Pins (Pins 5 to 10) [cite: 7, 42, 43]
#define enA 5
#define in1 6
#define in2 7
#define in3 8
#define in4 9
#define enB 10

// IR Sensors for Line Following [cite: 9, 10, 43, 44]
#define left_IR A0
#define right_IR A1

// Ultrasonic Sensor (HC-SR04) [cite: 11, 12, 44]
#define echoPin A2  // Referred to as "enable pin" in transcript
#define trigPin A3

// Servo Motor [cite: 13, 44]
#define servoPin A5

Servo myServo;

// --- Variables ---
int setpoint = 15; // Stop threshold in cm [cite: 15, 44]
long duration;
int distance_front, distance_left, distance_right;

void setup() {
  Serial.begin(9600); // Initialize Serial Monitor [cite: 17, 45]

  // Initialize Input Pins [cite: 18, 45]
  pinMode(left_IR, INPUT);
  pinMode(right_IR, INPUT);
  pinMode(echoPin, INPUT);

  // Initialize Output Pins [cite: 19, 45, 46, 47]
  pinMode(trigPin, OUTPUT);
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);
  pinMode(enB, OUTPUT);

  // Set motor speed to a controlled duty cycle (200 out of 255) [cite: 20, 47, 48]
  analogWrite(enA, 200);
  analogWrite(enB, 200);

  // Servo setup and initial sweep [cite: 21, 48]
  myServo.attach(servoPin);
  myServo.write(140);
  delay(300);
  myServo.write(0);
  delay(300);
  myServo.write(70); // Center position [cite: 30, 59]
  delay(500);
```

```
  // Take initial distance reading [cite: 21, 49]
  distance_front = ultra_read();
  delay(500); // Delay before main loop begins [cite: 22, 50]
}

void loop() {
  distance_front = ultra_read(); // Continuously read front distance [cite: 50]

  int left_sensor = digitalRead(left_IR);
  int right_sensor = digitalRead(right_IR);

  // Obstacle detection [cite: 16, 44, 45]
  if (distance_front < setpoint) {
    check_side(); // Stop and scan surroundings [cite: 27, 50, 51]
  }
  // Line following logic [cite: 24, 25, 51, 52, 53]
  else {
    if (left_sensor == 1 && right_sensor == 1) { // 1 generally means white surface/no line
      forward();
    } else if (left_sensor == 0 && right_sensor == 1) { // 0 means black line detected
      turn_left();
    } else if (
```

pinux_esp32.ino (Segment 2)

```
left_sensor == 1 && right_sensor == 0) {
      turn_right();
    } else {
      stop();
    }
  }

  delay(10); // 10-millisecond delay for continuous loop [cite: 23, 53]
}

// --- Custom Functions ---

// Transmit pulse and calculate distance in cm [cite: 26, 53, 54]
int ultra_read() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH);
  return duration * 0.034 / 2;
}

// Stop and scan left/right sides for obstacle avoidance [cite: 27, 57]
void check_side() {
  stop();
  delay(100);

  myServo.write(140); // Check right side [cite: 28, 57]
  delay(500);
  distance_right = ultra_read();
  Serial.print("Right: ");
  Serial.println(distance_right);

  myServo.write(0); // Check left side [cite: 29, 58, 59]
  delay(500);
  distance_left = ultra_read();
  Serial.print("Left: ");
  Serial.println(distance_left);

  myServo.write(70); // Return to center [cite: 30, 59]
  delay(500);
  compare_distance(); // Determine best path [cite: 30, 59]
}

// Execute bypass sequence based on clearance [cite: 31]
void compare_distance() {
  if (distance_left > distance_right) {
    // Left side has more clearance [cite: 32, 54, 55, 56]
    turn_left();
    delay(500);
    forward();
    delay(600);
    turn_right();
    delay(500);
    forward();
    delay(600);
    turn_right();
    delay(400);
  } else {
    // Right side has more clearance [cite: 33, 56, 57]
    turn_right();
    delay(500);
    forward();
    delay(600);
    turn_left();
    delay(500);
    forward();
    delay(600);
```

```
    turn_left();
    delay(400);
  }
}

// --- Motor Movement Functions ---

// Move Forward: clockwise HIGH, anti-clockwise LOW [cite: 34, 59, 60]
void forward() {
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);
}

// Move Backward: anti-clockwise HIGH, clockwise LOW [cite: 35, 60]
void backward() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  digitalWrite(in3, LOW);
  digitalWrite(in4, HIGH);
}

// Turn Right: Left motor clockwise, Right motor anti-clockwise [cite: 36, 60]
void turn_right() {
  digita
```

pinux_esp32.ino (Segment 3)

```
lWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, HIGH);
}

// Turn Left: Left motor anti-clockwise, Right motor clockwise [cite: 37, 60, 61, 62]
void turn_left() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);
}

// Stop: All directional motor pins to LOW [cite: 38, 62]
void stop() {
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  digitalWrite(in3, LOW);
  digitalWrite(in4, LOW);
}
```

10. Troubleshooting

Checklist for Deployment:

1. Ensure Firebase rules allow 'test_mode' or provide a valid API key.
2. Verify the 5V 2A power supply is sufficient for 8 relays firing simultaneously.
3. Check the ESP32 serial monitor (115200 baud) for WiFi connection logs.